

Author: Awet G. Nway

Senior Engineer — Robotics Maintenance & PLC Operations | ASAN Auto Parts Mfg., Gimpo, South Korea



DAY 7

LARGE LANGUAGE MODELS

The Engine Behind ChatGPT, Claude & Gemini — Explained Properly

A Course Guide by AYE Tech Hub

Author: Awet G. Nway

Senior Engineer — Robotics Maintenance & PLC Operations
ASAN Auto Parts Manufacturing, Gimpo, South Korea

Table of Contents

#	Section	Key Topics
1	Introduction & Learning Objectives	Why LLMs matter; what makes them different
2	The Transformer Architecture	Embeddings, attention, blocks, output layer
3	Self-Attention Explained	How LLMs understand context and references
4	How LLMs Are Trained	Pretraining → fine-tuning → RLHF
5	The Major LLMs of 2026	Claude, GPT, Gemini, open-source families
6	Context Windows & Tokens	How much an LLM can read at once
7	RAG — LLMs + Your Own Data	Connecting LLMs to private documents
8	Model Sizes: Small vs Frontier	Picking the right model for cost and capability
9	LLMs at the Workplace	Practical engineering applications
10	Summary, Self-Check Quiz & Homework	Recap + 5 questions + hands-on exercise

1. Introduction

In Day 6 you met Generative AI as a category. Today we go deep into the most important single technology inside it: the **Large Language Model**, or **LLM**. Every major AI assistant you have heard of — ChatGPT, Claude, Gemini, Copilot, DeepSeek, Llama, Mistral — is a Large Language Model. Understanding how they work, how they are trained, and how to choose between them is now a core engineering skill, like knowing the basics of databases or networking.

An LLM is, at heart, a giant next-word predictor — but the architecture that makes it work, the **Transformer**, is one of the most elegant inventions in computer science. First proposed in a 2017 Google paper titled *Attention Is All You Need*, the transformer replaced older sequence models and made it possible to train networks at unprecedented scale. By 2026 some models have over a trillion parameters and read entire books in one go. This lesson explains how that works in plain language.

Learning Objectives

By the end of this lesson you will be able to:

- Draw the high-level structure of a transformer (embedding, attention blocks, output).
- Explain self-attention with a real sentence example.
- Describe the three stages of LLM training: pretraining, supervised fine-tuning, RLHF.
- Compare the major LLM families (Claude, GPT, Gemini, open-source) and pick the right one for a task.
- Explain what tokens and context windows are, and why they affect cost and quality.
- Describe what RAG is and when to use it instead of fine-tuning.

2. The Transformer Architecture

Inside every modern LLM is the same basic architecture, called a **transformer**. Input text is broken into **tokens** (pieces of words), each token is converted into a long list of numbers (an **embedding**), and those numbers flow through a stack of **transformer blocks**. Each block does two things: it lets every token compare itself to every other token (**self-attention**) and then refines the result through a small neural network (the **feed-forward** layer). At the top, the model outputs a probability score for every possible next word — and the highest-scoring word is selected.

The Transformer Architecture — Engine of Every Modern LLM

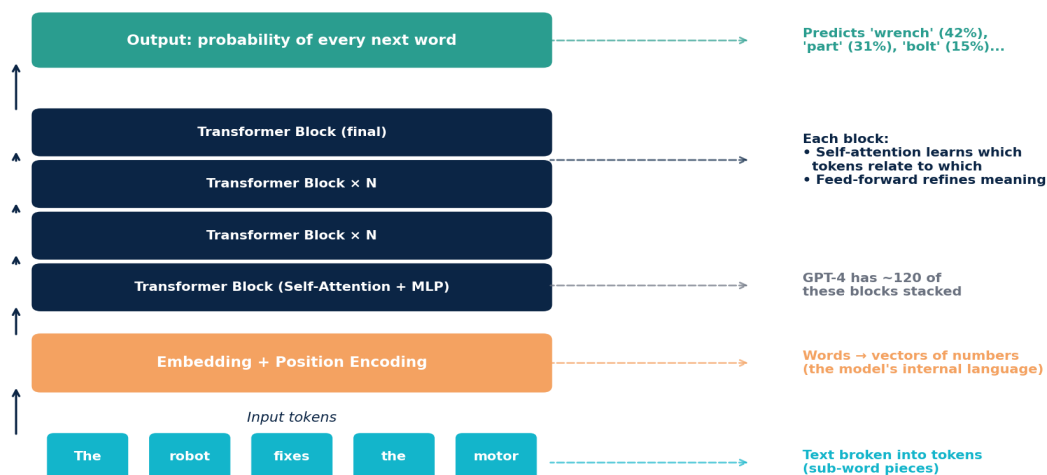


Figure 2.1 — The transformer architecture used in every modern LLM. Input flows from bottom to top through many identical blocks, ending with a prediction of the next word.

Scale Matters

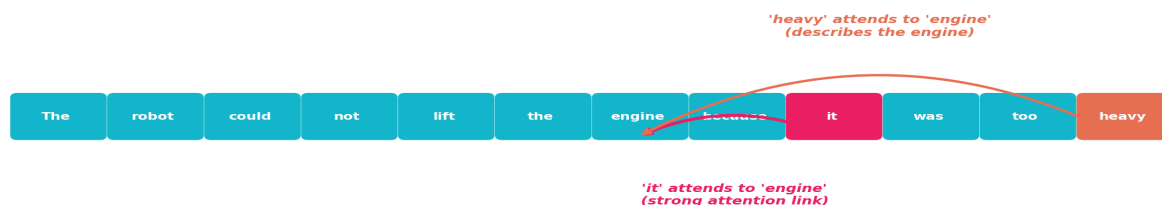
Modern frontier LLMs have **tens to hundreds of billions** of parameters (the weights inside the network). GPT-4 reportedly has over **1.7 trillion**. Each parameter is a single number — and all of them have been adjusted during training so the model produces useful output. The whole model is essentially just a giant function: text in, probabilities out. Everything ChatGPT and Claude do — code, poetry, reasoning, math — emerges from running this function billions of times.

3. Self-Attention — The Heart of the Transformer

The breakthrough that made transformers work is **self-attention**. Older models (RNNs, LSTMs) processed text word-by-word, left to right, which made them slow and bad at long-range understanding. Self-attention lets every token in a sentence **look at every other token simultaneously** and decide which ones are most relevant.

Consider the sentence: *The robot could not lift the engine because it was too heavy.* What does **it** refer to? Old systems often got this wrong. Self-attention solves it by letting **it** compute a similarity score against every previous token. The score with **engine** is highest, so the model correctly understands that **it** refers to the engine. The same mechanism handles long-range dependencies across paragraphs, code blocks, and entire documents.

Self-Attention — How a Transformer Knows Which Words Relate



Attention lets every word 'look at' every other word and weight their relevance — this is why transformers understand context so well.

Figure 3.1 — Self-attention links each word to the words that give it meaning. Here, 'it' attends to 'engine' (its true referent) and 'heavy' also attends to 'engine' (its description).

Multi-Head Attention

In practice, transformers run **multiple attention heads** in parallel — typically 8 to 128 of them per block. Each head learns a different kind of relationship: one might track grammatical roles, another might track who-did-what-to-whom, another might track topical similarity. The combined output gives the transformer its rich understanding of language. This is called **multi-head attention** and is one reason transformers are so much more powerful than what came before.

4. How an LLM Is Trained

A finished LLM like Claude or GPT does not simply emerge from a single training run. Modern LLMs are built in **three sequential stages**, each solving a different problem. Skip any stage and the model becomes either useless, dangerous, or both.

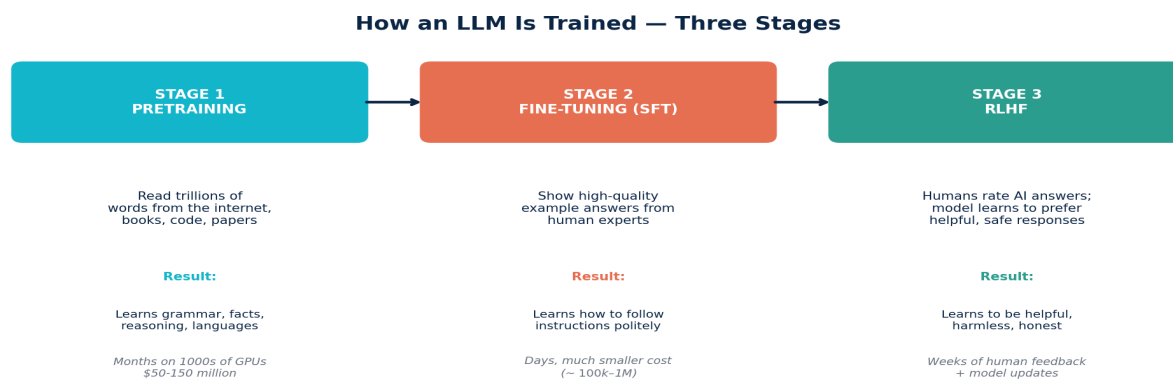


Figure 4.1 — The three stages of LLM training. Pretraining builds general knowledge, fine-tuning teaches instruction following, RLHF aligns the model with human preferences.

Stage	Data Used	What It Teaches	Time & Cost
1. Pretraining	Trillions of words from the internet, books, code	Language, facts, reasoning patterns	Months on thousands of GPUs; \$50–150 M+
2. Supervised Fine-Tuning (SFT)	Tens of thousands of expert-written prompt/answer pairs	Follow instructions politely and helpfully	Days; ~\$100k–\$1M
3. Reinforcement Learning from Human Feedback (RLHF)	Human ratings comparing pairs of AI responses	Be helpful, harmless, honest	Weeks of human labellers + model updates

Why Three Stages?

A model that only sees pretraining knows a lot of facts but cannot follow instructions — ask it 'summarise this email' and you might just get more email-like text. SFT teaches it to act like an assistant. But fine-tuned models still produce unsafe or unhelpful output sometimes. RLHF closes that gap by directly rewarding the answers humans actually prefer. ChatGPT's massive success in 2022 came largely from being the first model to combine all three stages at scale.

5. The Major LLMs of 2026

The LLM landscape changes monthly, but four families dominate the market in 2026. Each has different strengths, pricing, and design philosophies. Knowing which to use for which task is now a real engineering decision — like choosing between PLC brands or motor manufacturers.

The Major LLMs of 2026 — Who Makes What

Claude (Anthropic)	GPT-4 / GPT-5 (OpenAI)	Gemini (Google)	Llama / Open (Meta + others)
Variants: Opus, Sonnet, Haiku	Variants: Multiple variants	Variants: Pro, Flash, Ultra	Variants: Llama 3, Mistral, DeepSeek, Qwen
Best for: Reasoning, long docs, code	Best for: General-purpose, multimodal	Best for: Search-grounded, Google integration	Best for: Open-source, run locally

Figure 5.1 — The four major LLM families of 2026 and what each is known for.

Family	Maker	Open / Closed	Best Known For
Claude (Opus, Sonnet, Haiku)	Anthropic	Closed (API + apps)	Strong reasoning, long documents, code, safety
GPT (GPT-4, GPT-5, o-series)	OpenAI	Closed	General-purpose, multimodal, large ecosystem
Gemini (Pro, Flash, Ultra)	Google DeepMind	Closed	Google search integration, long context
Llama, Mistral, DeepSeek, Qwen	Meta + Mistral + DeepSeek + Alibaba	Open-source (run locally)	Free to host, private deployment, customisable

6. Context Windows & Tokens

An LLM cannot remember anything between separate conversations. Within one conversation, it can only 'see' what fits inside its **context window** — the maximum number of tokens it can read at once. A token is roughly 4 characters of text or about 0.75 of an English word. Longer context windows let the model read entire books, codebases, or datasets in one go.

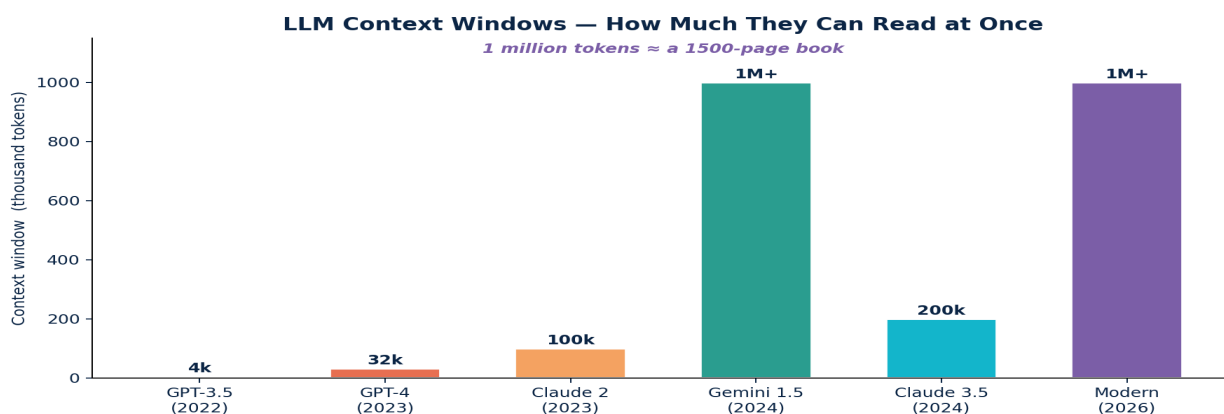


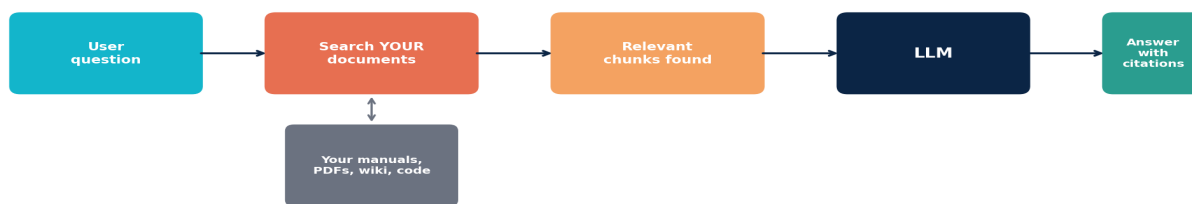
Figure 6.1 — Context window growth from GPT-3.5 (4 000 tokens) to modern LLMs (1 million+). 1 million tokens is roughly equivalent to a 1 500-page book.

7. RAG — Retrieval-Augmented Generation

What if you want an LLM to answer questions using **your** private documents — factory manuals, internal wikis, PLC code repositories — that the model never saw during training? You can either **fine-tune** the model on your data (expensive, slow) or use **RAG** (Retrieval-Augmented Generation), which is cheaper, faster, and works with any LLM. RAG searches your documents for relevant chunks at query time and feeds them to the

LLM as part of its context — letting it cite your sources in the answer.

Retrieval-Augmented Generation (RAG) — Give LLMs Your Own Data



RAG lets the LLM answer using YOUR private data without retraining the model.

Figure 7.1 — A RAG pipeline. The user's question triggers a search of YOUR documents; relevant chunks are passed to the LLM which generates a grounded, cited answer.

8. Model Sizes — Choosing the Right One

Not every task needs a frontier model. Smaller LLMs (7B to 70B parameters) are far cheaper, faster, and often run on a single laptop. Frontier models (400B+) are the only choice for the hardest reasoning, but cost 20× more per query. As an engineer you should match the model to the job — use small models for routine tasks (autocomplete, summary, translation) and frontier models only for complex reasoning, novel code, and difficult decisions.

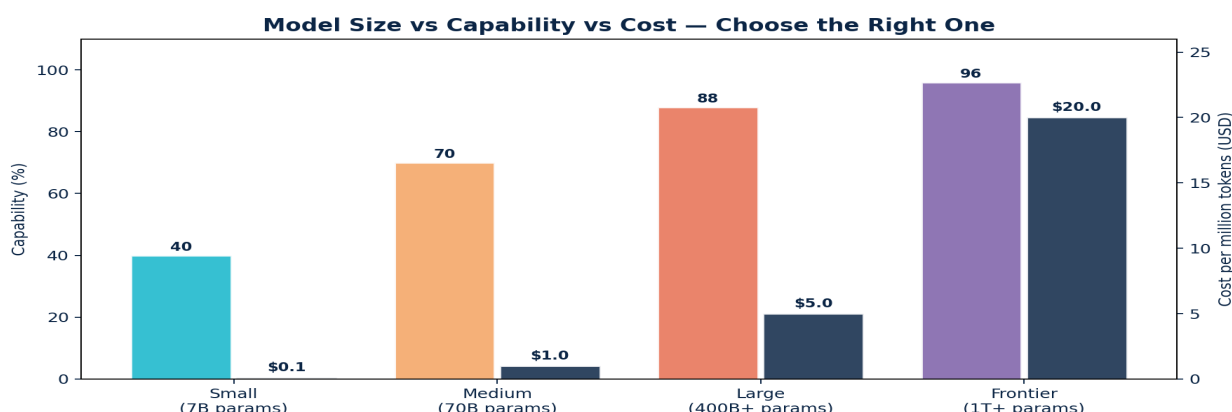


Figure 8.1 — As model size grows, capability rises but cost grows much faster. Most production work in 2026 uses medium-sized models; frontier models are reserved for high-value tasks.

9. LLMs at the Workplace

On the factory floor and the engineer's desk, LLMs are now used for dozens of daily tasks. The eight categories below cover most real-world engineering use in 2026 — and every one of them saves hours per week when used well.

Where Engineers Actually Use LLMs at Work

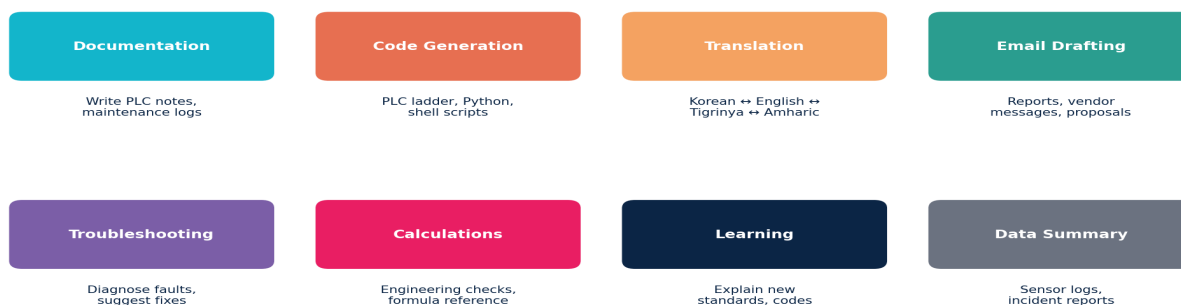


Figure 9.1 — Eight engineering categories where LLMs save real time at work. Documentation, code, and translation are the highest-impact starting points.

10. Summary, Self-Check Quiz & Homework

Lesson Recap

- An **LLM** is a giant transformer-based neural network that predicts the next word.
- **Transformer architecture** = embeddings → many self-attention + feed-forward blocks → output probabilities.
- **Self-attention** lets every token look at every other token in the input, which is why LLMs handle long-range context so well.
- **Training has three stages**: pretraining (read the internet), supervised fine-tuning (learn to follow instructions), RLHF (align with human preferences).
- Four major families in 2026: **Claude**, **GPT**, **Gemini**, and **open-source** (Llama, Mistral, DeepSeek, Qwen).
- **Context window** = how much text the model can read at once. Modern models reach 1 million+ tokens.
- **RAG** connects an LLM to your private documents without retraining it.
- Match the **model size** to the task — smaller models are cheaper and faster for routine work; frontier models only for complex reasoning.

Self-Check Quiz (answers at the bottom)

1. What is self-attention, and why was it a breakthrough?
2. Name the three stages of LLM training in order.
3. What does a context window measure, and roughly how many words is 1 million tokens?
4. When should you choose RAG over fine-tuning a model?
5. Why would you ever use a small (7B) LLM instead of a frontier (1T) model?

Answers: 1) A mechanism that lets every token in the input compare itself to every other token; it solved the long-range context problem and enabled training at massive scale. 2) Pretraining → Supervised Fine-Tuning (SFT) → RLHF. 3) The maximum number of tokens the model can read at once; 1 million tokens ≈ 750 000 English words ≈ a 1500-page book. 4) When you want the LLM to use YOUR private data without paying to retrain the whole model. 5) Cost, speed, and the ability to run locally — small models are 20× cheaper per query and good enough for routine tasks like summaries, translation, and autocomplete.

Homework Assignment

Pick one engineering task you do regularly — write a maintenance report, troubleshoot a PLC error, draft a procurement email, or document a machine — and complete the following:

1. Identify which **LLM family** (Claude, GPT, Gemini, open-source) you would use and explain why.
2. Choose between a **small or large** model size based on cost vs quality.
3. Write the **system prompt** that sets up the AI to do the task well (role, tone, format, constraints).
4. Identify **three pieces of context** the model needs from your workplace that it would not know on its own — these are the chunks RAG would have to provide.
5. Try it on Claude or ChatGPT and note one improvement you would make on the next iteration.

Submit to awetgknway@gmail.com with subject *AI Day 7 Homework — Your Name*.

Coming Up Tomorrow — Day 8

Computer Vision in Practice — how AI sees the world. You'll learn how cameras and CNNs combine to power face unlock, defect inspection, self-driving cars, medical imaging, and robot vision on the factory floor.



Continue Learning with AYE Tech Hub

Website: ayetechub.com

YouTube · Telegram · Instagram · LinkedIn · Threads: [@ayetechub](https://twitter.com/ayetechub)

TikTok: [@ayetechpro](https://www.tiktok.com/@ayetechpro) | Facebook: [ayetechengine](https://www.facebook.com/ayetechengine) & [ayetechpro](https://www.facebook.com/ayetechpro)

WhatsApp (Tigray Engineering Services): [+251919281449](tel:+251919281449)

Contact: awetgknway@gmail.com

Courses in **Tigrinya**, **Amharic** and **English**. Topics: AI · PLC · Automation · CAD/Revit · Electrical · Mechanical · HVAC · Solar